

链表的尾插法以及打印链表（与头插法混合版）

```
1 //这一块代码的讲解在王道C语言督学营[97]—16_3_链表头插法和遍历
2 #define _CRT_SECURE_NO_WARNINGS
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>
6 //先设计链表的某一个结点
7 typedef struct node_s { //也就是说这个struct node_s里不仅有一个数据data，还包含一个名为next的指针
8     //数据域：存储节点数据
9     int data;
10    //指针域：指向下一个节点的指针
11    struct node_s* next; //声明的指针变量next目前未初始化，未指向任何具体节点，值是随机的（垃圾值）
12 }node_t; //以上是在构造链表的“一个个结点”
13 typedef struct link_list_s {
14     node_t* phead; //头指针：指向链表第一个节点（phead是个指针变量
15     //存储内存地址，phead指向的结点才包含data和next成员）
16     node_t* ptail; //尾指针：指向链表最后一个节点
17 }link_list_t; //以上是在构造一个“链表整体”
18
19 //头插法（函数声明）
20 void head_insert(link_list_t* plist, int data); // *plist指向的位置包含node_t类型的指针变量phead和ptail
21 //用* plist是因为在我们做头插法时，phead和ptail的内容可能会变[若不用* plist，加引用&list也可（C++）]
22 //int data输入一个数据，根据数据创建链表结点，在查找链表头部
23
24 //打印链表（本质上就是遍历链表）（实际上只需要链表中的头指针就可以了）
25 void print_list(link_list_t* plist) { //这里其实可以不用传指针，以为打印链表并不会改变phead和ptail的指向
26     //但考虑到其他函数都传了指针，这边传一下也没事，不用传也没事
27     node_t* pcur = plist->phead; //这是个用来遍历整个链表的游标，一开始指向链表中的第一个结点的plist->phead
28     while (pcur != NULL) { //pcur在不停往后移，总有一天会变成空指针（最后一个节点的 next 指针是 NULL）
29         printf("%3d", pcur->data);
30         if (pcur->next != NULL) {
31             printf("->");
32         }
33         pcur = pcur->next;
34     }
35     printf("\n");
36 }
37
38 //尾插法
39 void tail_insert(link_list_t* plist, int data);
40
41 int main() {
42     link_list_t list; //准备好一个链表
43     //初始化
44     list.phead = NULL;
45     list.ptail = NULL;
46
47     //head_insert(&list, 1);
48     //print_list(&list);
49     //head_insert(&list, 3);
50     //print_list(&list);
51     //head_insert(&list, 5); //此时链表内容：5 -> 3 -> 1
52     //print_list(&list);
53
54     tail_insert(&list, 1);
55     print_list(&list);
56     tail_insert(&list, 3);
57     print_list(&list);
58     tail_insert(&list, 5);
59     print_list(&list);
60     return 0;
61 }
62 void head_insert(link_list_t* plist, int data) {
63     //1 给新结点“pnew_node”申请内存&&初始化（在“堆”上申请，否则“栈”会在函数结束以后销毁内存）
64     //堆上申请，void *malloc(size_t size);例子:int* arr = (int*)malloc(3 * sizeof(int));
65     node_t* pnew_node = (node_t*)malloc(sizeof(node_t)); //pnew_node是node_t类型指针变量，含data和next
66     pnew_node->next = NULL; //新结点的指针域一开始总是NULL（这样子也方便我们去操作）
67     pnew_node->data = data; //这两步分别在给data和next成员赋值
68     //然后我们要把这个新结点插到旧的链表里面（也就是plist所描述的链表里面）
69
70     //2 分类讨论（ptail的指向到底会不会变）
71     if (plist->phead == NULL) { //说明链表是空的
72         plist->phead = pnew_node;
73         plist->ptail = pnew_node; //phead、ptail两个指针都指向新结点
74     }
```

```

75     else {          //phead不为空，说明链表中原来至少有一个结点，这种情况下ptail指向都不会变
76         pnew_node->next = plist->phead; //让新结点pnew_node的指针域指向原来的第一个结点
77         //对应C语言讲义p46右上角的操作a，改变了指针的指向而非内容
78         plist->phead = pnew_node;    //把用来描述第一个结点的指针的值换成pnew_node
79         //这样的话我们就把链表中的新结点插到了链表的头部
80     }
81 }
82
83 void tail_insert(link_list_t* plist, int data) {
84     node_t* pnew_node = (node_t*)malloc(sizeof(node_t));
85     pnew_node->next = NULL;
86     pnew_node->data = data;
87
88     //2 分类讨论
89     if (plist->phead == NULL) {
90         plist->phead = pnew_node;
91         plist->ptail = pnew_node;
92     }
93     else {
94         plist->ptail->next = pnew_node; //第一个->:根据链表的地址找到ptail（找到尾指针ptail以后，ptail本身
95         //的指向不变，而是改变尾指针指向的结点的指针域，即改了next的指向）
96         //plist->ptail->next: plist链表中的尾指针ptail所指向的结点的指针域
97         plist->ptail = pnew_node;      //这里才是改了ptail的指向
98     }
99 }
100
101 //以上就是链表头插法的一个操作，示意图在p46页下方

```

